LegalEase: AI-Powered Legal Document Generator

Project Description:

LegalEase leverages state-of-the-art Generative AI to simplify the creation of legal documents by providing customizable, accurate, and editable templates for a wide range of use cases. Users can generate employment contracts, lease agreements, NDAs, freelance work contracts, and more, all tailored to their specific inputs like involved parties, effective dates, and key terms.

Built with FastAPI and integrated with a powerful AI core (Google Gemini), LegalEase ensures users receive professional-grade legal documents that maintain formatting standards, include custom branding (such as logos and fonts), and support features like editable previews. With seamless document export options (.PDF, .DOCX, .TXT), the platform bridges the gap between accessibility and legal professionalism.

LegalEase empowers entrepreneurs, professionals, and individuals to generate reliable legal content confidently, without needing a legal background.

Scenarios:

Scenario 1: A startup founder enters details about a new hire and selects "Employment Contract" as the document type. LegalEase generates a comprehensive agreement with clauses for roles, responsibilities, compensation, and confidentiality. The user customizes the final wording and downloads the document as a branded PDF.

Scenario 2: A freelancer needs a Non-Disclosure Agreement for a potential client. By specifying the parties, scope of confidentiality, and effective date, they instantly receive a structured NDA.

Scenario 3: A landlord uses LegalEase to draft a residential lease agreement. By inputting the property address, tenant details, and lease terms, the platform creates a formatted document with editable sections and a downloadable version in .DOCX format.

Pre-requisites

1. FastAPI Framework Knowledge: FastAPI Documentation (fastapi.tiangolo.com)
2. Google Gemini API Familiarity: Google Generative AI Documentation (ai.google.dev)
3. Streamlit for Frontend: Streamlit Documentation (docs.streamlit.io)
4. Python Programming Proficiency: Python Official Docs (docs.python.org)
5. Environment Setup: `pip install fastapi uvicorn streamlit python-docx fpdf2 Pillow requests google-generativeai python-dotenv`

Project Workflow

Activity 1: Model Selection and System Architecture

- **1.1: Research and select the Model**: Gemini 2.5 Flash (`gemini-2.5-flash`) from Google Generative AI for legal document generation.
- **1.2: Define architecture:**
  - Frontend: Streamlit for user input, preview, editing and document download.
  - Backend: FastAPI (main.py, routes.py) to handle API requests.
  - AI Integration: Gemini API used inside ai_core/gemini_generator.py.
- **1.3: Set up development environment** with a .env file containing the Gemini API key.

Activity 2: Core Functionalities Development

- **2.1: Implement core generation features:**
  - Legal document creation
  - Editable text preview
  - Format and export as .txt, .docx, and .pdf
- **2.2: Build FastAPI backend:**
  - Endpoint /generate to process POST requests
  - Accepts document_type, parties, terms, dates
  - Returns generated text from Gemini

Activity 3: main.py & API Logic

- **3.1: Create and configure main.py:**
  - Initialize FastAPI app
  - Route / for home (plus /health)
  - Include router from routes.py
- **3.2: Handle document generation logic in routes.py:**
  - Define Pydantic model DocumentRequest
  - Call GeminiDocumentGenerator.generate_document(...)

Activity 4: Frontend with Streamlit

- **4.1: Design user-friendly Streamlit interface:**
  - Input fields for all required data
  - Responsive layout with title and logo
- **4.2: Dynamic document generation preview:**
  - Show styled editable output
  - Option to export as .txt, .docx, or .pdf

Activity 5: Deployment

- **5.1: Prepare deployment:**
  - Install dependencies from requirements.txt
  - Set up Procfile, requirements.txt, or Dockerfile if needed
- **5.2: Deploy application:**
  - Local: uvicorn (backend) + streamlit (frontend), or the run.sh helper
  - Cloud options: Render, Railway, Fly.io or a VPS for the backend; Streamlit Community Cloud for the frontend

---

Activity 6: Conclusion

- **Finalize and test:**
  - Ensure end-to-end functionality
  - Test Gemini integration under different input combinations

MILESTONE 1: Model Selection and Architecture – LegalEase

In this milestone, we focus on selecting the optimal generative AI model for creating personalized and accurate legal documents using Google's Gemini model via the Google Generative AI SDK. We also define the system architecture and set up the development environment to support document generation, editing, and exporting in various formats (.docx, .pdf, .txt).

Activity 1.1: Research and Select the Appropriate Generative AI Model

1. **Understand the Project Requirements**
   - LegalEase must generate structured legal documents (e.g., contracts, NDAs, agreements) based on user inputs such as:
   - Document type
   - Parties involved
   - Terms and conditions
   - Effective dates
2. **Explore Gemini Model Documentation**
   - Visit the Google Generative AI Documentation to evaluate the available Gemini models.
   - Review the capabilities of gemini-2.5-flash, especially: long context window, structured prompting, and fast generation speed.
3. **Evaluate Model Performance**
   - Gemini 2.5 Flash handles complex formatting, reasoning, and structured text generation with low latency, making it suitable for drafting legal content interactively.
4. **Select the Optimal Model**

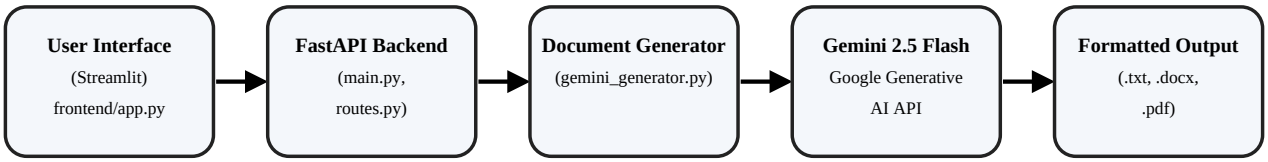
   Selected: `gemini-2.5-flash`

   - Well suited for drafting formal documents with clear sections and legal wording.
   - Generates clauses dynamically from the user's inputs.
   - The model name is read from the GEMINI_MODEL variable in .env, so it can be changed without touching the code.

Activity 1.2: Define the Architecture of the Application

1. Architectural Diagram

- Components:
  - **Frontend**: Streamlit for user interaction (document input, preview, editing, downloads)
  - **Backend**: FastAPI app (routes input to the Gemini model and returns generated content)
  - **AI Core**: Gemini integration for content generation
  - **Formatting Modules**: DOCX, PDF, TXT generators




```
Component interaction
-----------------------------------------------------------------------------------------------------------

+---------------------------+        +-------------------------------+
|     Streamlit Frontend    | <----> |        FastAPI Backend        |
|                           |        |                               |
| - User inputs             |        | - Receives JSON payload       |
| - Document preview        |        | - Calls Gemini model          |
| - Edit + download buttons |        | - Returns legal content       |
+---------------------------+        +---------------+---------------+
                                                     |
                                                     v
                                     +-------------------------------+
                                     |     AI Document Generator     |
                                     |  (Gemini 2.5 Flash via SDK)   |
                                     +-------------------------------+
```

2. Frontend Functionality

- Users provide inputs via:
  - TextInput for document type and effective date
  - TextArea for parties and terms
- Buttons:
  - "Generate Document"
  - "Click to Edit Document"
  - "Download as .TXT / .DOCX / .PDF"
- Styled HTML preview of the generated legal document

3. Backend Responsibilities

- Expose POST /generate endpoint (via FastAPI)
- Accept JSON input (document_type, parties, terms, dates)
- Route to GeminiDocumentGenerator
- Return structured document text

4. AI Integration Points

- GeminiDocumentGenerator forms a structured prompt and sends it to Gemini via generate_content
- Receives generated text and returns it to FastAPI
- Frontend displays the text and allows editing or exporting

Activity 1.3: Set Up the Development Environment

1. **Install Python and Pip**
   Ensure Python 3.10+ is installed along with pip for dependency management (python.org/downloads). Install VS Code (code.visualstudio.com) and get a Gemini API key from aistudio.google.com/app/apikey.
2. **Create and activate a Virtual Environment** (open the VS Code terminal with Ctrl + `)

```
Windows (PowerShell)
-----------------------------------------------------------------------------------------------------
python -m venv venv
venv\Scripts\activate
```

```
macOS / Linux
-----------------------------------------------------------------------------------------------------
python3 -m venv venv
source venv/bin/activate
```

You should see (venv) at the start of the terminal prompt.

3. **Install all dependencies** (FastAPI, Uvicorn, Streamlit, Google Generative AI SDK, python-docx, fpdf2, Pillow, requests, python-dotenv) from requirements.txt:

```
pip install -r requirements.txt
```

or individually:

```
pip install fastapi uvicorn streamlit python-docx fpdf2 Pillow requests google-generativeai python
    -dotenv
```

4. **Set Up Application Structure** – create the folder LegalEase and recreate the structure below.

```
Project structure
-----------------------------------------------------------------------------------------------------
LegalEase/
|
|-- .env
|-- .gitignore
|-- requirements.txt
|-- run.sh
|-- config.py
|
|-- ai_core/
|   |-- __init__.py
|   `-- gemini_generator.py
|
|-- legalEaseAPI/
|   |-- __init__.py
|   |-- main.py
|   `-- routes.py
|
|-- frontend/
|   |-- __init__.py
|   |-- app.py
|   `-- utils/
|       |-- __init__.py
|       |-- formatters.py
|       `-- sanitizer.py
```

```
|
|-- Image/
|   |-- Logo.png
|   `-- inverseLogo.png
|
`-- docs/
```

The Image folder holds Logo.png and inverseLogo.png. If no logo is available, the app falls back to a text header.

Configuration files

```
requirements.txt
---------------------------------------------------------------------------------------------------
fastapi==0.115.0
uvicorn[standard]==0.32.0
streamlit==1.40.0
python-docx==1.1.2
fpdf2==2.8.2
Pillow==10.4.0
requests==2.32.3
google-generativeai==0.8.3
python-dotenv==1.0.1
pydantic==2.9.2
```

```
.env
---------------------------------------------------------------------------------------------------
GEMINI_API_KEY=YOUR_GEMINI_API_KEY_HERE
GEMINI_MODEL=gemini-2.5-flash
BACKEND_URL=http://localhost:8000
```

Replace YOUR_GEMINI_API_KEY_HERE with your actual Gemini API key from aistudio.google.com/app/apikey. Never commit this file; it is listed in .gitignore.

```
.gitignore
---------------------------------------------------------------------------------------------------
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
venv/
env/
.venv/

# Environment
.env

# IDE
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db

# Logs
*.log
```

```
# Build
build/
dist/
*.egg-info/

# Streamlit
.streamlit/secrets.toml
```

```
config.py
---------------------------------------------------------------------------------------------------------
import os
from dotenv import load_dotenv

load_dotenv()

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
BACKEND_URL = os.getenv("BACKEND_URL", "http://localhost:8000")

# Paths
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
IMAGE_DIR = os.path.join(BASE_DIR, "Image")
LOGO_PATH = os.path.join(IMAGE_DIR, "Logo.png")
INVERSE_LOGO_PATH = os.path.join(IMAGE_DIR, "inverseLogo.png")
```

MILESTONE 2: Core Functionalities Development

Activity 2.1: Develop the core functionalities

1. Legal Document Generation

Functionality: Generate legal documents (e.g., contracts, agreements) based on user inputs (document type, parties, terms, and dates). The route passes the four inputs to the generator, which returns the document text.

```python
# routes.py
@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates,
    )
    return {"document": response}
```

```python
# gemini_generator.py
genai.configure(api_key=api_key)
self.model = genai.GenerativeModel(model_name=self.model_name)
response = self.model.generate_content(prompt)
```

The complete generator class, including the prompt, is shown below. The prompt asks Gemini for formal legal language, a clear title, standard sections (Parties, Effective Date, Terms & Conditions, Confidentiality, Termination, Governing Law, Entire Agreement, Severability, Signature block), numbered clauses converted from the semicolon-separated terms, placeholders such as [Address] for missing information, and a branded footer line.

```
ai_core/__init__.py
```

```python
from .gemini_generator import GeminiDocumentGenerator

__all__ = ["GeminiDocumentGenerator"]
```

```
ai_core/gemini_generator.py
```

```python
import os
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()


class GeminiDocumentGenerator:
    """Handles prompt construction and Gemini API calls for legal document generation."""

    def __init__(self, model_name: str = None):
        api_key = os.getenv("GEMINI_API_KEY")
        if not api_key:
            raise ValueError(
                "GEMINI_API_KEY not found. Please set it in your .env file."
            )
```

```
        genai.configure(api_key=api_key)
        self.model_name = model_name or os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
        self.model = genai.GenerativeModel(model_name=self.model_name)

    def generate_document(
        self,
        document_type: str,
        parties: str,
        terms: str,
        dates: str,
    ) -> str:
        """Generate a legal document using Gemini based on user inputs."""

        prompt = f"""You are a professional legal document drafting assistant.
Generate a comprehensive, formal legal document of type: "{document_type}".

Document Details:
- Document Type: {document_type}
- Involved Parties: {parties}
- Effective Date: {dates}
- Terms and Conditions: {terms}

Requirements:
1. Use formal legal language and structure.
2. Include a clear title at the top.
3. Include sections such as: Parties, Effective Date, Terms & Conditions,
   Confidentiality, Termination, Governing Law, Entire Agreement, Severability,
   and Signature block.
4. Convert semicolon-separated terms into numbered clauses.
5. Use placeholders (e.g., [Address], [State]) where information is missing.
6. Ensure the output is clean, well-formatted plain text with numbered
   headings and proper spacing.
7. Do NOT include markdown code fences, asterisks, or emojis.
8. End with a signature block and a footer line: "LegalEase Inc. | contact@legalease.com | All Rig
    hts Reserved."

Return ONLY the document text.
"""

        try:
            response = self.model.generate_content(prompt)
            text = getattr(response, "text", None)
            if not text:
                # Fallback for safety-blocked or empty responses
                raise RuntimeError("Gemini returned an empty response.")
            return text.strip()
        except Exception as e:
            raise RuntimeError(f"Gemini generation failed: {str(e)}")
```

2. Document Output Formatting

Functionality: Convert raw AI-generated text into structured .docx, .pdf, .txt files with legal formatting, logos, and footers.

- `sanitize_text(text)`
- `format_docx(text, doc_type)`
- `format_pdf(text, doc_type)`
- `format_html_preview(text)`

These use:

- **python-docx** for Microsoft Word formatting (includes logo, Times New Roman font, headings, footer).
- **fpdf2** for clean, branded PDF documents with header/footer on every page.
- **HTML Preview** with Streamlit styling (dark card, highlighted headings).

frontend/utils/__init__.py

```
from .sanitizer import sanitize_text
from .formatters import format_docx, format_pdf, format_html_preview

__all__ = ["sanitize_text", "format_docx", "format_pdf", "format_html_preview"]
```

frontend/utils/sanitizer.py

```
import re


def sanitize_text(text: str) -> str:
    """Clean AI output: normalize quotes, strip markdown artifacts, tidy spacing."""
    if not text:
        return ""

    # Replace typographic quotes with plain ones
    replacements = {
        "\u2018": "'", "\u2019": "'",
        "\u201c": '"', "\u201d": '"',
        "\u2013": "-", "\u2014": "-",
        "\u2026": "...", "\u00a0": " ",
    }
    for k, v in replacements.items():
        text = text.replace(k, v)

    # Remove markdown code fences
    text = re.sub(r"```[a-zA-Z]*", "", text)
    text = text.replace("```", "")

    # Collapse 3+ blank lines into 2
    text = re.sub(r"\n{3,}", "\n\n", text)

    # Trim trailing spaces
    text = "\n".join(line.rstrip() for line in text.splitlines())

    return text.strip()
```

frontend/utils/formatters.py

```
import os
import re
import io
from datetime import datetime

from docx import Document
from docx.shared import Pt, Inches, RGBColor
from docx.enum.text import WD_ALIGN_PARAGRAPH
from fpdf import FPDF

from config import LOGO_PATH


# ============================================================
```

```python
#  DOCX FORMATTER
# ============================================================
def format_docx(text: str, doc_type: str) -> bytes:
    """Create a formatted DOCX with logo, title, terms table, and footer."""
    document = Document()

    # Page styling
    style = document.styles["Normal"]
    style.font.name = "Times New Roman"
    style.font.size = Pt(11)

    # Logo at the top
    if os.path.exists(LOGO_PATH):
        try:
            logo_para = document.add_paragraph()
            logo_para.alignment = WD_ALIGN_PARAGRAPH.CENTER
            logo_para.add_run().add_picture(LOGO_PATH, width=Inches(1.2))
        except Exception:
            pass

    # Title
    title = document.add_heading(doc_type or "Legal Document", level=0)
    title.alignment = WD_ALIGN_PARAGRAPH.CENTER

    # Body text
    for line in text.split("\n"):
        stripped = line.strip()
        if not stripped:
            document.add_paragraph("")
            continue

        # Detect headings like "1. Services:" or "Section 1"
        if re.match(r"^(\d+\.|Section\s+\d+|ARTICLE\s+\w+)", stripped, re.IGNORECASE):
            h = document.add_heading(stripped, level=2)
            h.alignment = WD_ALIGN_PARAGRAPH.LEFT
        else:
            document.add_paragraph(stripped)

    # Footer with page number
    section = document.sections[0]
    footer = section.footer
    footer_para = footer.paragraphs[0]
    footer_para.alignment = WD_ALIGN_PARAGRAPH.CENTER
    footer_para.text = (
        f"LegalEase Inc. | contact@legalease.com | "
        f"Generated on {datetime.now().strftime('%Y-%m-%d')}"
    )

    buffer = io.BytesIO()
    document.save(buffer)
    buffer.seek(0)
    return buffer.getvalue()


# ============================================================
#  PDF FORMATTER
# ============================================================
class _LegalPDF(FPDF):
    def __init__(self, doc_type: str):
        super().__init__()
        self.doc_type = doc_type
        self.set_auto_page_break(auto=True, margin=20)

    def header(self):
```

```
        if os.path.exists(LOGO_PATH):
            try:
                self.image(LOGO_PATH, x=85, y=8, w=40)
                self.ln(28)
            except Exception:
                pass
        self.set_font("Times", "B", 16)
        self.cell(0, 10, self.doc_type or "Legal Document", align="C", ln=True)
        self.ln(4)

    def footer(self):
        self.set_y(-15)
        self.set_font("Times", "I", 8)
        self.cell(
            0,
            10,
            f"LegalEase Inc. | contact@legalease.com | Page {self.page_no()}",
            align="C",
        )


def _clean_latin(text: str) -> str:
    """fpdf2 core fonts are latin-1; strip unsupported unicode."""
    replacements = {
        "\u2018": "'", "\u2019": "'",
        "\u201c": '"', "\u201d": '"',
        "\u2013": "-", "\u2014": "-",
        "\u2026": "...", "\u00a0": " ",
        "\u2022": "-",
    }
    for k, v in replacements.items():
        text = text.replace(k, v)
    return text.encode("latin-1", "replace").decode("latin-1")


def format_pdf(text: str, doc_type: str) -> bytes:
    """Create a branded PDF with logo, headings, and footer."""
    pdf = _LegalPDF(doc_type)
    pdf.add_page()
    pdf.set_font("Times", size=11)

    for line in text.split("\n"):
        stripped = _clean_latin(line.strip())
        if not stripped:
            pdf.ln(4)
            continue

        if re.match(r"^(\d+\.|Section\s+\d+|ARTICLE\s+\w+)", stripped, re.IGNORECASE):
            pdf.set_font("Times", "B", 12)
            pdf.multi_cell(0, 7, stripped)
            pdf.set_font("Times", size=11)
        else:
            pdf.multi_cell(0, 6, stripped)

    out = pdf.output(dest="S")
    if isinstance(out, str):
        out = out.encode("latin-1", "replace")
    return bytes(out)


# ============================================================
#  HTML PREVIEW FORMATTER
# ============================================================
def format_html_preview(text: str) -> str:
```

```
    """Convert plain text into styled HTML for the Streamlit dark-themed preview."""
    if not text:
        return ""

    html_lines = []
    for line in text.split("\n"):
        stripped = line.strip()
        if not stripped:
            html_lines.append("<br/>")
            continue

        safe = (
            stripped.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
        )

        if re.match(r"^(\d+\.|Section\s+\d+|ARTICLE\s+\w+)", stripped, re.IGNORECASE):
            html_lines.append(
                f"<h4 style='color:#7dd3fc; margin-top:16px; margin-bottom:6px;'>{safe}</h4>"
            )
        else:
            html_lines.append(
                f"<p style='color:#e2e8f0; margin:4px 0; line-height:1.5;'>{safe}</p>"
            )

    return "".join(html_lines)
```

3. User Interaction

Functionality: Accept user input through the Streamlit interface and reflect it in the AI-generated legal content.

```
# app.py - Streamlit UI
document_type = st.text_input("Document Type")
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions (Use semicolons for bullet points)")
dates = st.text_input("Effective Date")

# Send to backend
response = requests.post(f"{BACKEND_URL}/generate", json={
    "document_type": document_type,
    "parties": parties,
    "terms": terms,
    "dates": dates,
})
```

4. Editable Document & Download

Functionality: Provides an editable text area and allows download in .txt, .docx, .pdf formats.

```
# app.py
if st.session_state.show_edit:
    edited_text = st.text_area(
        "Edit Document Below:",
        value=st.session_state.generated_text,
        height=300,
        key="edit_area",
    )
    st.session_state.generated_text = edited_text
```

```
# Downloads
st.download_button(label="Download as .TXT", data=st.session_state.generated_text.encode("utf-8"),
     ...)
st.download_button(label="Download as .DOCX", data=format_docx(...), ...)
st.download_button(label="Download as .PDF", data=format_pdf(...), ...)
```

Activity 2.2: Implement the FastAPI Backend

1. Define Routes in FastAPI

The router is included in main.py with `app.include_router(router)` and the route `@router.post("/generate")` generates the legal document.

2. Process User Input

Handled via Pydantic and Streamlit forms: the frontend collects the input and FastAPI validates it with the DocumentRequest model (document_type, parties, terms and dates, all strings).

3. Integrate Gemini API Calls

The generator configures the SDK with the key from .env and calls `generate_content(prompt)`. Empty or blocked responses raise an error that the route converts to an HTTP 500 response.

MILESTONE 3: API Logic Integration

In Milestone 3, the focus shifts to designing and integrating the backend API logic that powers document generation using the Gemini model. This milestone ensures a functional bridge between frontend inputs and the AI-powered document generation, which is the core of the LegalEase project.

Activity 3.1: Create and Configure main.py

This step involves initializing the FastAPI application, setting up base endpoints for health-checks, enabling CORS so Streamlit (on a different port) can call the API, and including modular API routes.

Steps:

- Initialize the FastAPI app
- Define a root (/) endpoint (and /health) to verify the service is running
- Enable CORS middleware
- Include routes from the routes.py module

```
legalEaseAPI/__init__.py
-------------------------------------------------------------------------------------------------------
# Package marker for legalEaseAPI
```

```
legalEaseAPI/main.py
-------------------------------------------------------------------------------------------------------
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from legalEaseAPI.routes import router

app = FastAPI(
    title="LegalEase - AI Legal Document Generator",
    description="Backend API for generating AI-powered legal documents.",
    version="1.0.0",
)

# CORS so Streamlit (different port) can call this API
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Include API routes
app.include_router(router)


@app.get("/")
def home():
    return {"message": "Welcome to LegalEase AI Legal Document Generator API"}


@app.get("/health")
def health():
    return {"status": "ok"}
```

```
if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Activity 3.2: Handle Document Generation Logic in routes.py

This task handles user inputs from the frontend and sends them to the Gemini model for document generation.

Steps:

- Define a Pydantic DocumentRequest model to validate incoming data.
- Initialize the GeminiDocumentGenerator from ai_core.gemini_generator.
- Create a POST endpoint (/generate) to accept user input and return generated legal content, returning HTTP 500 with the error detail if generation fails.

```
legalEaseAPI/routes.py
-----------------------------------------------------------------------------------------------------------
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
gemini_generator = GeminiDocumentGenerator()


class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str


@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    try:
        response = gemini_generator.generate_document(
            request.document_type,
            request.parties,
            request.terms,
            request.dates,
        )
        return {"document": response}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

MILESTONE 4: Frontend Development – LegalEase

Milestone 4 focuses on creating a responsive and user-centric web interface for **LegalEase** using **Streamlit**. This milestone enables users to easily input legal details, generate AI-powered documents, preview the results, make inline edits, and download documents in multiple formats (TXT, DOCX, PDF) with rich formatting and embedded branding.

Activity 4.1: Designing and Developing the User Interface

Step 1: Streamlit Page Configuration and Layout Setup

- Configured the Streamlit app with:

```
st.set_page_config(
    page_title="LegalEase",
    page_icon="",
    layout="centered",
)
```

- Added a three-column layout using st.columns to center-align the company logo (with a text fallback):

```
col1, col2, col3 = st.columns([1, 2, 1])
with col2:
    if os.path.exists(LOGO_PATH):
        st.image(LOGO_PATH, use_container_width=True)
```

Step 2: Header and Title

- Displayed a center-aligned title under the logo:

```
st.markdown(
    "<h2 style='text-align: center; color: #7dd3fc;'>AI Legal Document Generator</h2>",
    unsafe_allow_html=True,
)
```

Step 3: User Input Interface

- Created intuitive inputs, wrapped in an st.form, for:
  - document_type: type of legal document
  - parties: involved stakeholders
  - terms: semicolon-separated clauses
  - dates: effective date

Activity 4.2: Creating Dynamic Previews and Export Options

Step 1: Generate Document with Backend AI

- On clicking "Generate Document", a POST request is sent to the FastAPI backend and the AI-generated content is sanitized and stored in session state:

```
response = requests.post(f"{BACKEND_URL}/generate", json={...}, timeout=120)
...
st.session_state.generated_text = sanitize_text(data["document"])
```

Step 2: HTML Preview Rendering

- The output is previewed in a scrollable card with semantic HTML (headings and paragraphs):

```
styled_html = format_html_preview(st.session_state.generated_text)
st.markdown(f"<div style='...'>{styled_html}</div>", unsafe_allow_html=True)
```

Step 3: Editable Document Preview

- The user can click "Click to Edit Document" and modify the generated content using a text area:

```
edited_text = st.text_area("Edit Document Below:", value=st.session_state.generated_text, height=3
    00)
```

Step 4: Multi-Format Download Options

- Users can download the document as:
  - .txt (Plain Text)
  - .docx (Formatted Word Document with logo, headings, and footer)
  - .pdf (Branded PDF with logo and footer)

Core Functions & Utilities

sanitize_text(text)

- Normalizes typographic quotes and dashes, removes markdown code fences, collapses extra blank lines and trims trailing spaces.

format_docx(text, doc_type)

- Uses python-docx to embed the logo, add the title, detect numbered headings, format paragraphs and add a footer.

format_pdf(text, doc_type)

- Utilizes FPDF2 with custom header/footer: center-aligned logo, document title, bold headings for sections, and page numbers.

format_html_preview(text)

- Converts output to stylized HTML blocks for inline display (HTML-escaped for safety).

Complete frontend source

```
frontend/__init__.py
--------------------------------------------------------------------------------------------------
# Package marker for frontend
```

```
frontend/app.py
--------------------------------------------------------------------------------------------------
import os
import sys
import requests
```

```
import streamlit as st

# Make project root importable when running `streamlit run frontend/app.py`
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)

from config import BACKEND_URL, LOGO_PATH, INVERSE_LOGO_PATH
from frontend.utils import (
    sanitize_text,
    format_docx,
    format_pdf,
    format_html_preview,
)

# ---------------- Page Config ----------------
st.set_page_config(
    page_title="LegalEase",
    page_icon="⚖",
    layout="centered",
)

# ---------------- Session State ----------------
if "generated_text" not in st.session_state:
    st.session_state.generated_text = ""
if "show_edit" not in st.session_state:
    st.session_state.show_edit = False
if "doc_type" not in st.session_state:
    st.session_state.doc_type = "Legal Document"

# ---------------- Header / Logo ----------------
col1, col2, col3 = st.columns([1, 2, 1])
with col2:
    if os.path.exists(LOGO_PATH):
        st.image(LOGO_PATH, use_container_width=True)
    else:
        st.markdown(
            "<h1 style='text-align:center;'>⚖LegalEase</h1>",
            unsafe_allow_html=True,
        )

st.markdown(
    "<h2 style='text-align: center; color: #7dd3fc;'>AI Legal Document Generator</h2>",
    unsafe_allow_html=True,
)
st.markdown(
    "<p style='text-align:center; color:#94a3b8;'>"
    "Generate professional legal documents in seconds with Gemini 2.5 Flash."
    "</p>",
    unsafe_allow_html=True,
)
st.divider()

# ---------------- Input Form ----------------
with st.form("legal_form"):
    document_type = st.text_input(
        "Document Type",
        placeholder="e.g., Freelance Work Contract, NDA, Lease Agreement",
    )
    parties = st.text_area(
        "Parties Involved",
        placeholder="e.g., Jane Doe (Service Provider), TechNova Inc. (Client)",
        height=100,
    )
```

```
    terms = st.text_area(
        "Terms & Conditions (Use semicolons for bullet points)",
        placeholder="Payment within 30 days; Deliverables by deadline; "
        "Confidentiality maintained; 15-day termination notice",
        height=140,
    )
    dates = st.text_input(
        "Effective Date",
        placeholder="e.g., April 15, 2025",
    )

    submitted = st.form_submit_button("Generate Document", use_container_width=True)

# ---------------- Generate ----------------
if submitted:
    if not document_type or not parties or not terms or not dates:
        st.warning("Please fill in all fields before generating.")
    else:
        with st.spinner("Generating your legal document with Gemini 2.5 Flash..."):
            try:
                response = requests.post(
                    f"{BACKEND_URL}/generate",
                    json={
                        "document_type": document_type,
                        "parties": parties,
                        "terms": terms,
                        "dates": dates,
                    },
                    timeout=120,
                )
                if response.status_code == 200:
                    data = response.json()
                    st.session_state.generated_text = sanitize_text(data["document"])
                    st.session_state.doc_type = document_type
                    st.session_state.show_edit = False
                    st.success("Document Generated Successfully!")
                else:
                    st.error(
                        f"Backend error {response.status_code}: {response.text}"
                    )
            except requests.exceptions.ConnectionError:
                st.error(
                    f"Could not connect to backend at {BACKEND_URL}. "
                    "Make sure FastAPI (uvicorn) is running."
                )
            except Exception as e:
                st.error(f"Error: {str(e)}")

# ---------------- Preview ----------------
if st.session_state.generated_text:
    st.divider()
    st.markdown("### Document Preview")

    styled_html = format_html_preview(st.session_state.generated_text)
    st.markdown(
        f"""
        <div style="
            background-color:#0f172a;
            border:1px solid #1e293b;
            border-radius:10px;
            padding:22px;
            max-height:520px;
            overflow-y:auto;
            font-family:'Times New Roman', serif;
```

```
        ">
            {styled_html}
        </div>
        """,
        unsafe_allow_html=True,
    )

    # ---------------- Edit ----------------
    if st.button("✏Click to Edit Document"):
        st.session_state.show_edit = not st.session_state.show_edit

    if st.session_state.show_edit:
        edited_text = st.text_area(
            "Edit Document Below:",
            value=st.session_state.generated_text,
            height=300,
            key="edit_area",
        )
        st.session_state.generated_text = edited_text

    # ---------------- Downloads ----------------
    st.markdown("### Download Options")
    col_a, col_b, col_c = st.columns(3)

    with col_a:
        st.download_button(
            label="⬇Download as .TXT",
            data=st.session_state.generated_text.encode("utf-8"),
            file_name=f"{st.session_state.doc_type.replace(' ', '_').lower()}.txt",
            mime="text/plain",
            use_container_width=True,
        )

    with col_b:
        try:
            docx_bytes = format_docx(
                st.session_state.generated_text, st.session_state.doc_type
            )
            st.download_button(
                label="⬇Download as .DOCX",
                data=docx_bytes,
                file_name=f"{st.session_state.doc_type.replace(' ', '_').lower()}.docx",
                mime="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
                use_container_width=True,
            )
        except Exception as e:
            st.error(f"DOCX error: {e}")

    with col_c:
        try:
            pdf_bytes = format_pdf(
                st.session_state.generated_text, st.session_state.doc_type
            )
            st.download_button(
                label="⬇Download as .PDF",
                data=pdf_bytes,
                file_name=f"{st.session_state.doc_type.replace(' ', '_').lower()}.pdf",
                mime="application/pdf",
                use_container_width=True,
            )
        except Exception as e:
            st.error(f"PDF error: {e}")

# ---------------- Footer ----------------
```

```
st.divider()
st.markdown(
    "<p style='text-align:center; color:#64748b; font-size:12px;'>"
    "LegalEase Inc. | contact@legalease.com | All Rights Reserved."
    "</p>",
    unsafe_allow_html=True,
)
```

Note: emoji icons that appear in the UI labels of app.py (page icon, buttons and status messages) are omitted from this listing for font compatibility; the code itself is unchanged.

MILESTONE 5: Deployment

In Milestone 5, the focus is on deploying the **LegalEase** application locally using **FastAPI** as the backend and **Streamlit** as the frontend. This milestone involves setting up a virtual environment, configuring dependencies, and verifying that all modules, from the Gemini API integration to document generation and downloads, work seamlessly in a real-world local environment. This paves the way for cloud deployment in future stages.

Activity 5.1: Preparing the Application for Local Deployment

Step 1: Set Up a Virtual Environment

To ensure environment isolation and consistent dependency management, create and activate a virtual environment:

```
Windows (PowerShell)
-----------------------------------------------------------------------------------------------------
python -m venv venv
venv\Scripts\activate
```

```
macOS / Linux
-----------------------------------------------------------------------------------------------------
python3 -m venv venv
source venv/bin/activate
```

Step 2: Install Dependencies

Ensure that the backend (FastAPI) and frontend (Streamlit) dependencies are installed from requirements.txt:

```
pip install -r requirements.txt
```

Step 3: Configure Environment Variables

Create a .env file in the project root to securely store your **Gemini API key** (see Milestone 1).

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY_HERE
GEMINI_MODEL=gemini-2.5-flash
BACKEND_URL=http://localhost:8000
```

Activity 5.2: Testing and Verifying Local Deployment

Step 1: Launch the FastAPI Backend (Terminal 1)

Start the backend server using **Uvicorn**:

```
uvicorn legalEaseAPI.main:app --reload --port 8000
```

Expected output:

```
INFO:     Uvicorn running on http://127.0.0.1:8000
INFO:     Application startup complete.
```

Open http://localhost:8000 in the browser; it should return:

```
{"message": "Welcome to LegalEase AI Legal Document Generator API"}
```

Step 2: Launch the Streamlit Frontend (Terminal 2)

Open a new terminal in VS Code, activate the venv again, then run the app.py file to start the **LegalEase** UI:

```
streamlit run frontend/app.py
```

Expected output:

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.x.x:8501
```

URLs to open in the browser

Service	URL	Purpose
Frontend (main app)	http://localhost:8501	LegalEase UI: enter data, generate, preview, download
Backend API root	http://localhost:8000	Health / welcome message
Backend Swagger docs	http://localhost:8000/docs	Interactive API testing
Backend ReDoc	http://localhost:8000/redoc	Alternative API docs

Step 3: Test All Functionalities

- Input document type, parties, terms, and date.
- Press **Generate Document** to get AI-generated legal content from Gemini.
- Use built-in options to:
  - Preview the styled document
  - Edit the text
  - Download in .txt, .docx, and .pdf formats
- Check for:
  - Proper layout with logo and footer
  - Clean formatting of numbered clauses
  - Correct embedding of the entered terms in the document

Test input used

Field	Value
Document Type	Freelance Work Contract
Parties	Jane Doe (Service Provider), TechNova Inc. (Client)
Terms	Payment within 30 days; Deliverables by agreed deadline; Confidentiality maintained; 15-day termination notice
Effective Date	April 15, 2025

Quick backend test (optional)

The endpoint can also be tested directly, for example from PowerShell:

```
curl -X POST http://localhost:8000/generate `
  -H "Content-Type: application/json" `
  -d "{\"document_type\":\"NDA\",\"parties\":\"Alice, Bob\",\"terms\":\"Confidentiality; 2-year te
      rm\",\"dates\":\"2025-04-15\"}"
```

Optional helper script (Linux / macOS)

run.sh starts both the FastAPI backend and the Streamlit frontend. Windows users can simply run the two commands above in two terminals.

```
run.sh
-----------------------------------------------------------------------------------------------------
#!/usr/bin/env bash
# Starts both FastAPI backend and Streamlit frontend.

set -e

echo "Starting FastAPI backend on http://localhost:8000 ..."
uvicorn legalEaseAPI.main:app --reload --port 8000 &
BACKEND_PID=$!

sleep 3

echo "Starting Streamlit frontend on http://localhost:8501 ..."
streamlit run frontend/app.py &
FRONTEND_PID=$!

trap "kill $BACKEND_PID $FRONTEND_PID" EXIT
wait
```

Troubleshooting

Problem	Fix
ModuleNotFoundError: config	Run Streamlit from the project root: `streamlit run frontend/app.py`
GEMINI_API_KEY not found	Confirm .env is in the project root and python-dotenv is installed.
Connection refused in UI	The backend (uvicorn) is not running. Start Terminal 1.
404 model not found	Ensure GEMINI_MODEL=gemini-2.5-flash and that your API key has access to it.
PDF shows garbled characters	The PDF formatter maps text to latin-1; the sanitizer already replaces typographic characters. Keep custom text ASCII.
Logo not showing	Ensure Image/Logo.png exists (PNG, ideally about 300 x 300 px).

Exploring LegalEase

The screenshots below were captured from the running application with the Freelance Work Contract test input.

1. Document Type

Purpose: Type of legal document to generate.
Examples: Agreement, Contract, NDA (Non-Disclosure Agreement), Lease Agreement, Employment Offer Letter.
Example input: *Freelance Work Contract*

2. Parties Involved

Purpose: Names and roles of the individuals or entities involved in the document.
Examples: John Doe (Freelancer), ABC Corp (Client); Alice Smith (Tenant), XYZ Realty (Landlord).
Example input: *Jane Doe (Service Provider), TechNova Inc. (Client)*

3. Terms & Conditions (Use semicolons for bullet points)

Purpose: Specific clauses or rules that define the agreement. Separate each bullet point using a semicolon.
Example input: *Payment within 30 days; Deliverables by deadline; Confidentiality maintained*

4. Effective Date

Purpose: The date when the agreement becomes legally valid.
Example input: *15-04-2025 (April 15, 2025)*

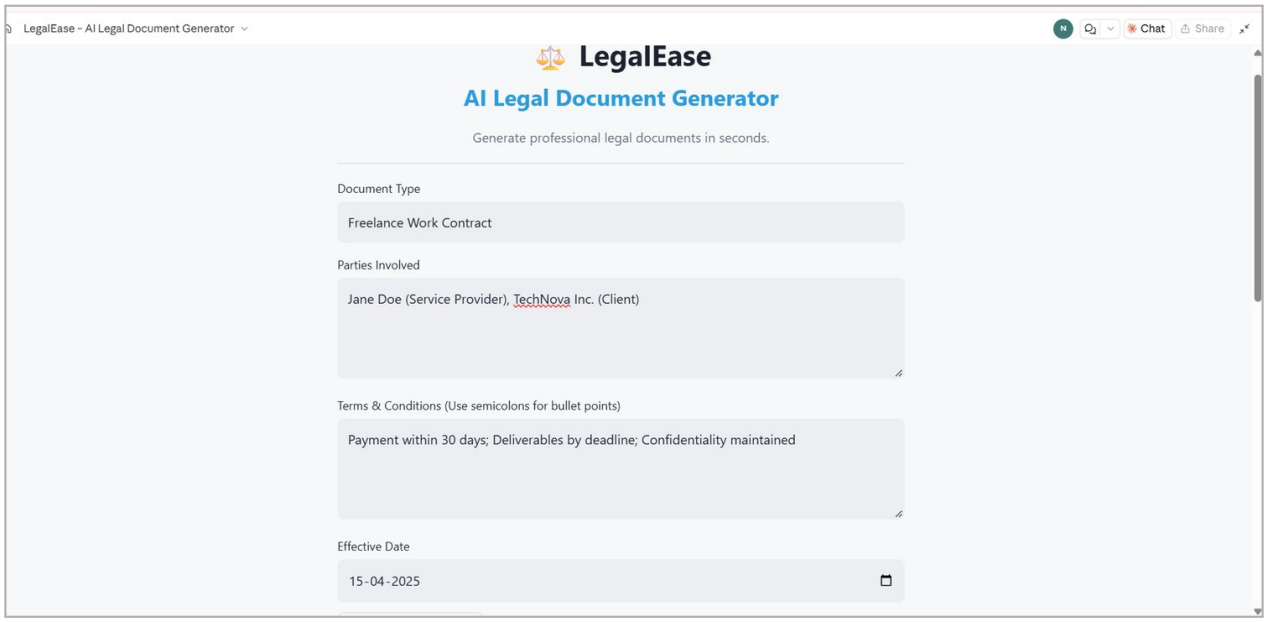


Figure 1: LegalEase input form filled with the Freelance Work Contract details.

Document Generated

After clicking **Generate Document**, the request is sent to the FastAPI backend, Gemini drafts the contract and the frontend shows the **Document Generated Successfully!** banner with a scrollable preview card. Below the preview are the **Click to Edit Document** button and the **Save your document** section with the TXT, DOCX and PDF download buttons.

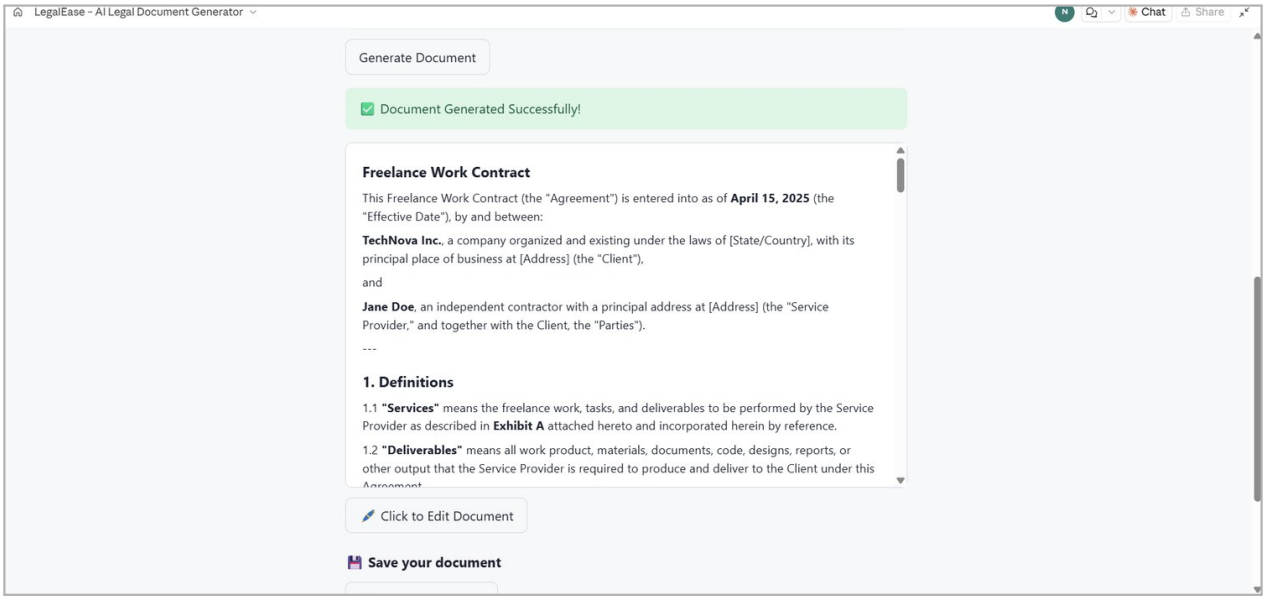


Figure 2: Success message and the start of the preview: title, effective date (April 15, 2025), the two parties and Section 1 (Definitions).

Reading the Generated Contract

Scrolling through the preview shows the full structure produced by the prompt, with numbered sections and sub-clauses and placeholders such as [Address], [State/Country] and [Interest Rate] where the user did not provide details.

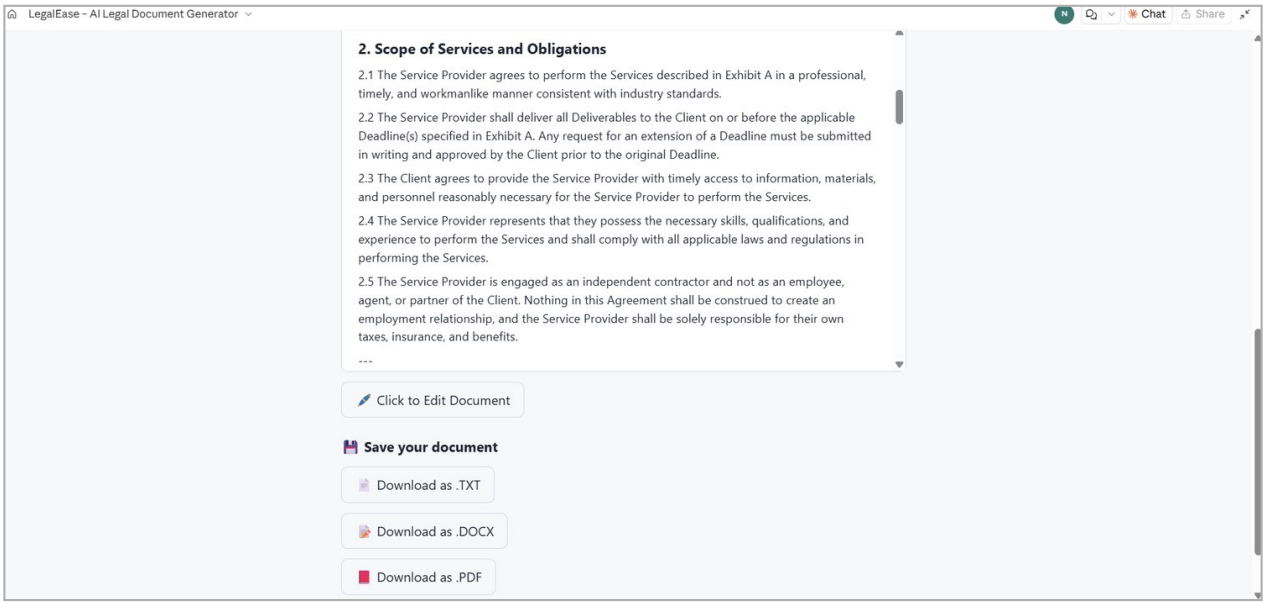


Figure 3: Section 2, Scope of Services and Obligations (clauses 2.1 to 2.5), with the edit and download buttons below the preview.

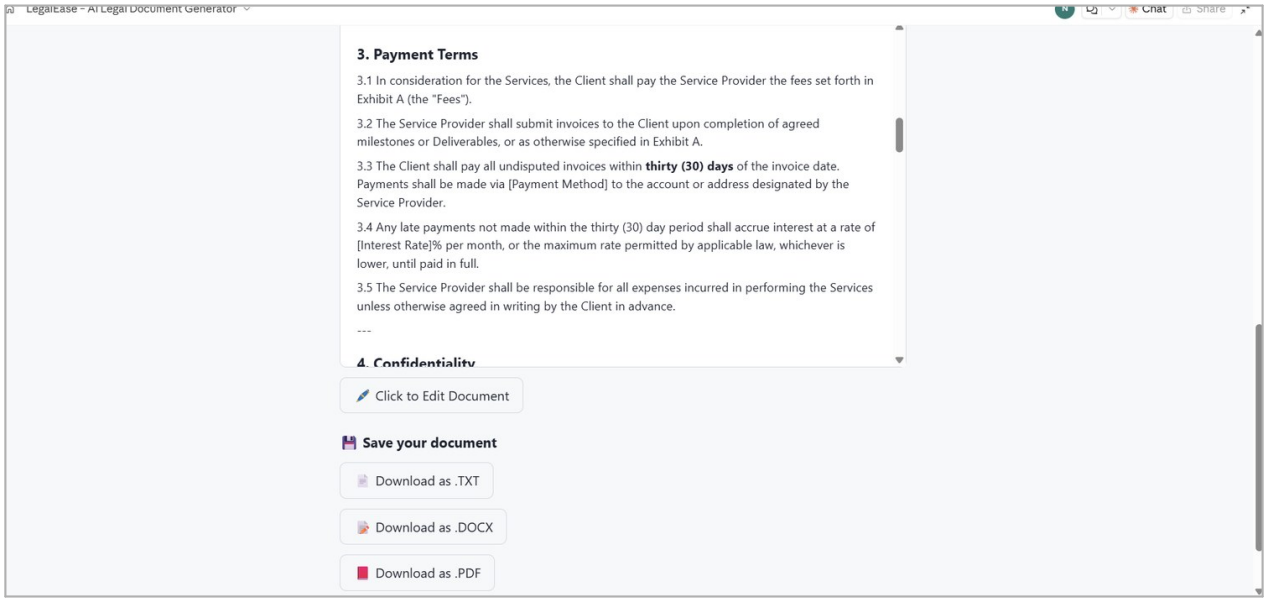


Figure 4: Section 3, Payment Terms: invoicing, payment within thirty (30) days, late-payment interest and expenses.

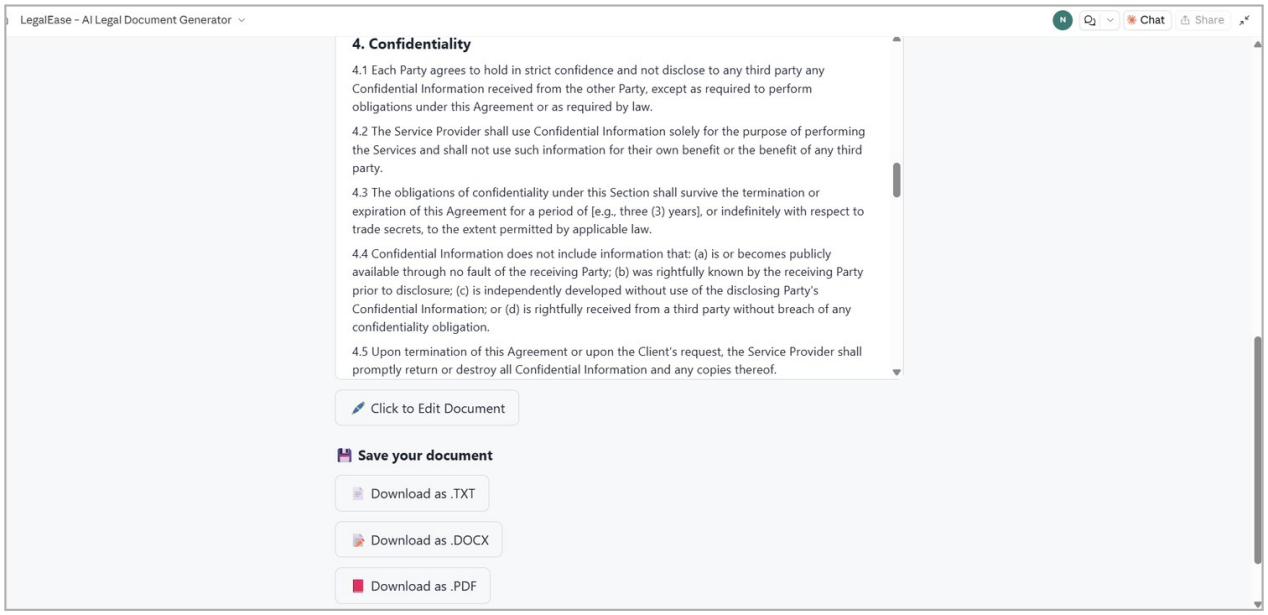


Figure 5: Section 4, Confidentiality: obligations, survival period, exclusions and return or destruction of information.

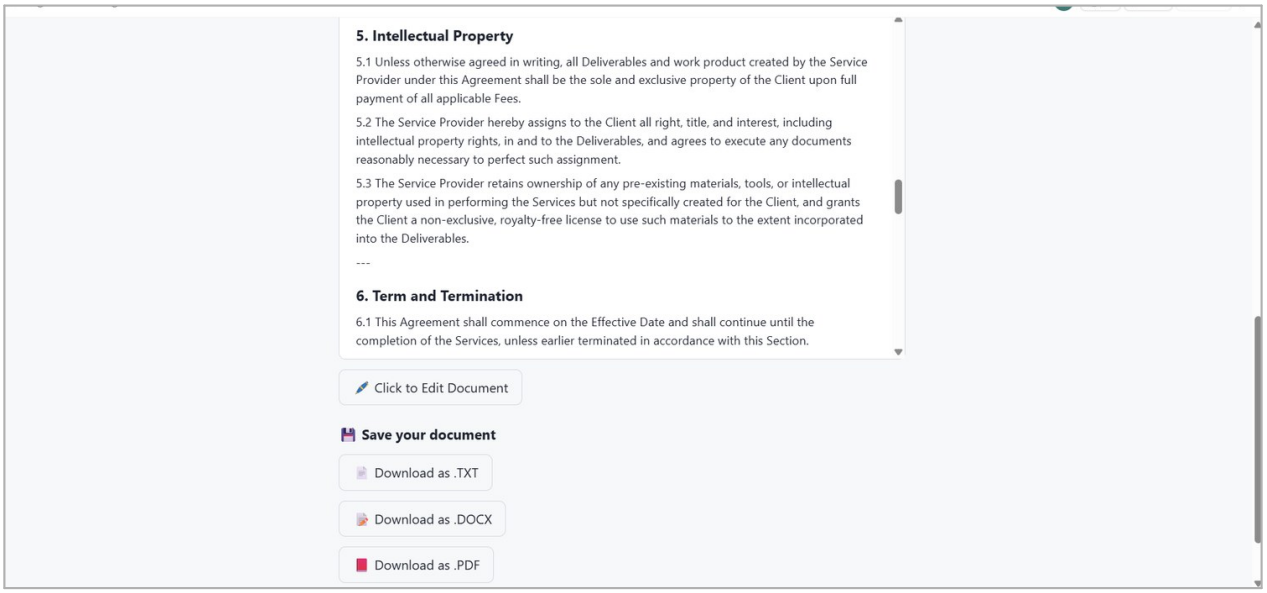


Figure 6: Section 5, Intellectual Property, and the beginning of Section 6, Term and Termination.

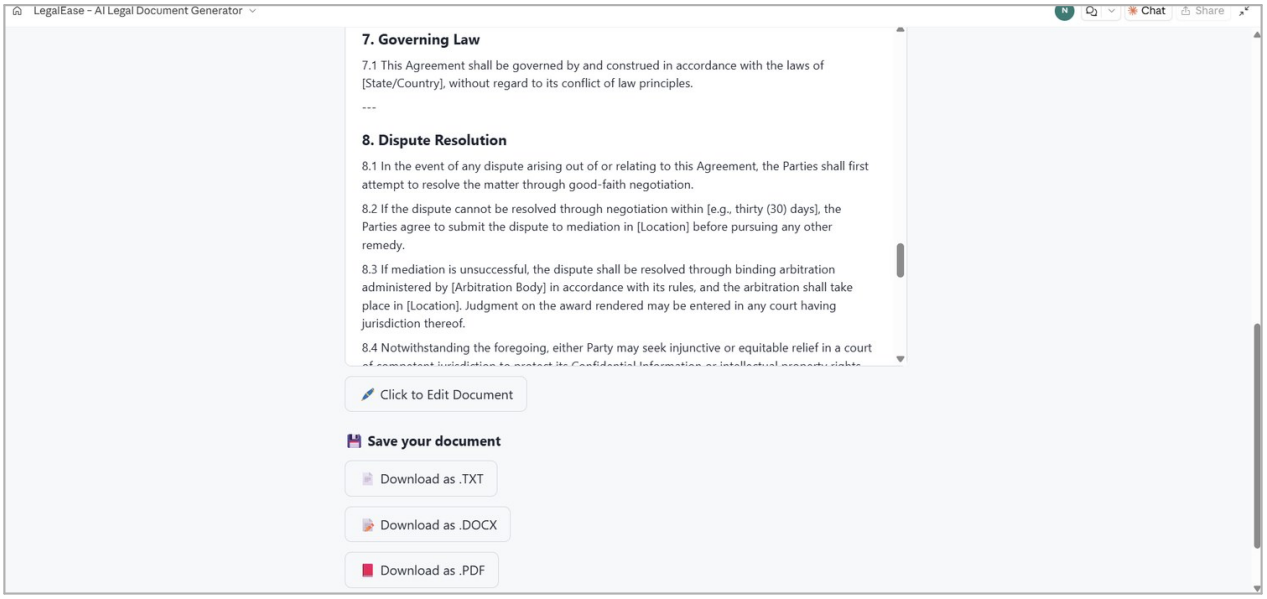


Figure 7: Section 7, Governing Law, and Section 8, Dispute Resolution (negotiation, mediation, arbitration).

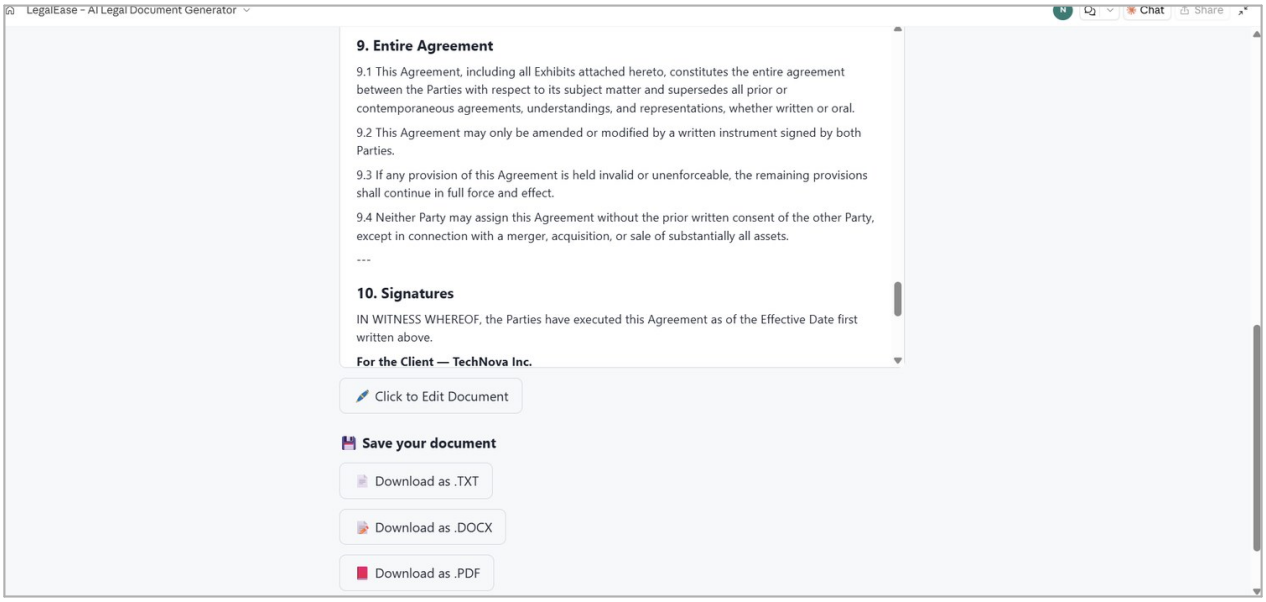


Figure 8: Section 9, Entire Agreement, and the start of Section 10, Signatures.

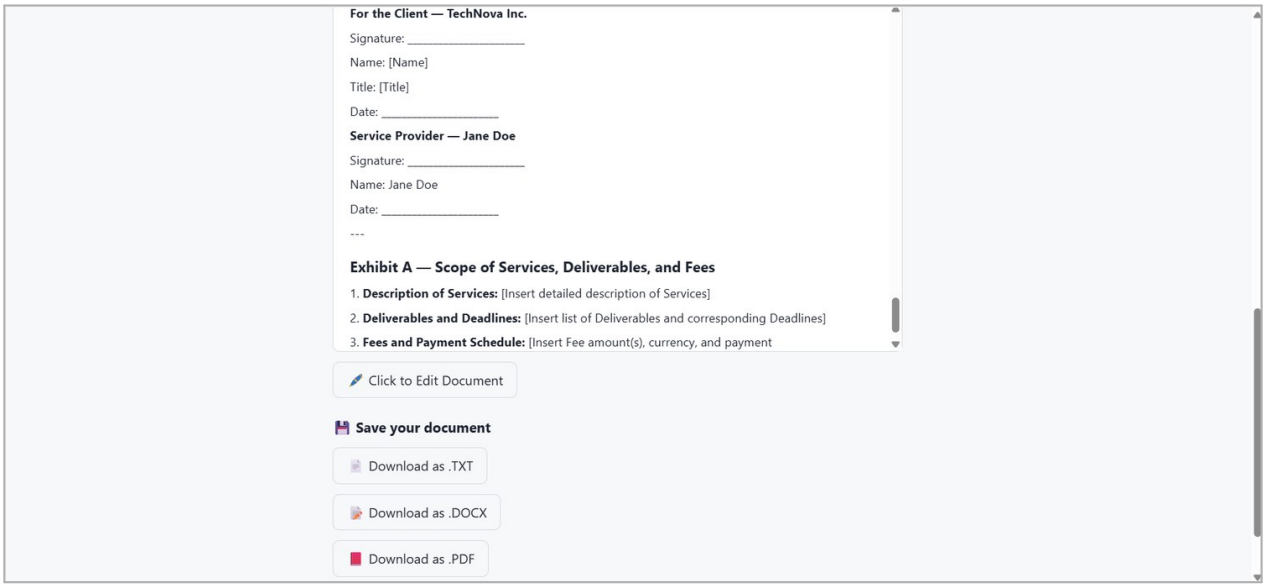


Figure 9: Signature blocks for the Client (TechNova Inc.) and the Service Provider (Jane Doe), followed by Exhibit A, Scope of Services, Deliverables, and Fees.

Editing the Document

If the user wants to edit the document, they can click the **Click to Edit Document** button. An *Edit Document Below:* text area opens containing the full text; any change made there is stored in session state and is used by all three download formats.

Downloading / Saving the Generated Document

The generated document can be downloaded and saved in any of the following formats:

- **.txt**: the plain edited text.
- **.docx**: a Word document with the logo at the top, a centered title, numbered headings, Times New Roman body text and a footer ("LegalEase Inc. | contact@legalease.com | Generated on ...").

- **.pdf**: a branded PDF with the logo and title in the header of each page, bold section headings, and a footer with the page number.

Conclusion

LegalEase shows how generative AI can make first drafts of legal documents faster and more approachable. A user supplies only four inputs (document type, parties, terms and effective date), and the application returns a structured, numbered agreement that can be reviewed in a styled preview, edited in place, and exported as TXT, DOCX or PDF.

From a technical standpoint the project separates concerns cleanly: a Streamlit frontend for interaction, a FastAPI backend that validates requests with Pydantic, an AI core that builds the prompt and calls the Gemini API, and formatting utilities that sanitize the output and produce branded documents. The model and backend address are configured through environment variables, which keeps the API key out of the source code.

Generated documents are drafts. They contain placeholders that must be completed and should be reviewed by a qualified legal professional before being relied on. Future enhancements could include deeper contract analysis, an automatically generated terms table from the semicolon-separated input, additional document templates, and cloud deployment of the backend and frontend.